

Guión práctica 1

Guion: Práctica 1 Programación Orientada a Objetos

Presentador: Bienvenidos a la explicación de la Práctica 1 de Programación Orientada a Objetos. En esta práctica se buscó, primero, comprender los fundamentos de Java implementando una calculadora sencilla que resuelve adición, sustracción, producto, cociente, potencia, raíz y módulo; y segundo, utilizar la asistencia de una LLM para identificar las ideas que nos proporciona sobre cómo recrear una versión primitiva de un programa tipo Paint, evaluando su desempeño y complejidad. También se buscó implementar un historial de las acciones ejecutadas usando estructuras de datos. Empezamos por el marco teórico.

Orador 1: Para entrenar a la IA fue necesario darle un contexto, así que le preguntamos cómo se puede generar un paint en el lenguaje de Java. Un paint es un método utilizado para dibujar elementos gráficos en una ventana: líneas, círculos, texto, imágenes y figuras geométricas, entre otros. Paint() le dice al programa qué debe dibujar en una superficie gráfica, y se utiliza mucho en Java AWT/Swing para crear gráficos y aplicaciones visuales, animaciones, videojuegos 2D sencillos y visualizar datos.

Presentador: Para generar la calculadora en Java se utilizaron diferentes librerías y algunas estructuras de control para que el código fuera más fácil de leer.

Orador 2: Usamos dos clases principales. java.util es una clase de utilidad general para el programador; de ahí viene, por ejemplo, la clase Scanner, utilizada para la entrada por teclado de diferentes tipos de datos, o la clase Date, para el tratamiento de fechas. Por su parte, java.math contiene herramientas para manipulaciones matemáticas.

Presentador: El siguiente bloque teórico fueron las estructuras de control, que le dan la lógica de flujo al programa.

Orador 3: La función if es la más básica: evalúa una condición y ejecuta un bloque de instrucciones en caso de que sea verdadera. La instrucción if-else agrega un segundo camino para cuando la condición es falsa. La estructura if-else if es parecida a las dos anteriores, con la diferencia de que se emplean varias condiciones encadenadas. Y la estructura switch se utiliza para seleccionar un bloque de instrucciones según el valor de una expresión: se evalúa la expresión, se compara con los valores de cada caso, y si hay coincidencia se ejecuta el bloque asociado; las sentencias break y default son opcionales.

Presentador: Con esa base teórica, pasamos al desarrollo. Siguiendo las indicaciones del profesor, se formularon varias consultas a distintos LLM para explorar cómo se crearía un programa similar a Paint, donde se elige una figura línea, cuadrado, círculo o rombo, se decide si es sólida (se colorea el interior) o solo el contorno, se

elige el color, y hay dos opciones de borrado: solo la figura anterior o borrar toda la pantalla.

Orador 4: De esas consultas surgieron dos enfoques. El primero, orientado a objetos, define una clase base abstracta *Figura* con atributos como las coordenadas, el color y si es sólida, y un método abstracto para dibujar; de ahí se desprenden subclases como *Linea*, *Rectangulo*, *Circulo* y *Rombo*. El lienzo administra una lista de estas figuras: para borrar la figura anterior se elimina el último elemento de la lista, y para borrar la pantalla se vacía la lista por completo y se redibuja. El segundo enfoque usa una matriz de caracteres en consola, apoyándose en una matriz de respaldo para poder deshacer el trazo anterior, sin depender de librerías externas de interfaz gráfica. Al comparar las respuestas, la diferencia principal entre los modelos fue el formato y la información: uno dio el algoritmo detalladamente paso a paso, otro dio un enfoque más técnico con fragmentos de código sin quitar la explicación, y otro optó por una descripción más teórica, enfocándose en los detalles técnicos del programa.

Presentador: Ahora, la segunda parte del desarrollo: la calculadora en Java.

Orador 5: El programa se ejecuta en el método estático *main*. Mediante la clase *Scanner* de *java.util* se leen los datos desde la consola. El flujo principal se organiza con una estructura de selección múltiple *switch-case*, con sentencias *break* y cláusula *default*, para despachar las operaciones aritméticas, apoyándose en el método estático *Math.pow* para el cálculo exponencial y radicular. También se incorporan estructuras condicionales *if-else* como validaciones para interceptar divisiones o residuos con divisor cero y raíces de base negativa, evitando indeterminaciones y resultados no numéricos, es decir, NaN. Al final, la ejecución concluye con el cierre explícito del flujo de entrada mediante *scan.close()*.

Presentador: Así lo comprobamos en las pruebas de validación: al ingresar 12 y 8 y seleccionar la operación de suma, la calculadora responde con el resultado de la suma: 20.0.

Presentador: Para cerrar, el primer objetivo de la práctica se cumplió, ya que se evaluó el algoritmo proporcionado por los LLM para un programa tipo *Paint* y se analizó en equipo, descubriendo diferencias entre las respuestas de cada modelo. El segundo objetivo se cumplió con el desarrollo de una calculadora en Java, donde en la captura de datos se usó la clase *Scanner* de la biblioteca *java.util*, y se emplearon estructuras de control como el *switch* para las siete opciones del menú, y las condicionales *if* e *if-else* para validar las operaciones, previniendo errores como la división entre cero.